

claude-session-publisher — Benutzerhandbuch

transcript_archiver.py v2.8.0

Contents

claude-session-publisher — Benutzerhandbuch	1
1. Schnellstart	2
2. Quellen	2
3. Referenz der Kommandozeile	2
Positionsargument	2
Auffinden und Ablage	3
Inhalt	3
Index	4
claude.ai-Import	4
Ausgabesteuerung und Protokollierung	4
4. Was erzeugt wird	5
Dateien	5
Die Seite	5
Sprache	5
Referenzmarken	6
Treuebericht	6
Verbrauch und Kosten	6
Die Bedienelemente der HTML-Seite	6
Der Index	7
5. LaTeX und PDF	7
6. Protokollierung und Prüfung	8
7. Bekannte Einschränkungen	8
8. Tests	9
9. Dieses Handbuch bauen	9

claude-session-publisher — Benutzerhandbuch

[English](#) · [Português \(Brasil\)](#) · [Español](#) · **Deutsch** · [Français](#)

Übersetzung des englischen Handbuchs, das die Referenz bleibt; Befehle, Dateinamen, Optionen und Codeblöcke stehen wie im Original.

`transcript_archiver.py` verwandelt ein Claude-Gespräch in ein in sich geschlossenes Dokument — HTML, reiner Text, Markdown, LaTeX oder PDF — mit einem Treuebericht, der jeden Quelldatensatz mit dem abgleicht, was die Seite zeigt. Dieses Handbuch ist die vollständige Referenz: jede Option, jede Ausgabe, jede Funktion und jede bekannte Einschränkung. Das README ist die Produktseite; `AGENTS.md` enthält dieselbe Information, geschrieben für einen KI-Agenten, der das Werkzeug bedient.

Eine Datei, Python 3.9+, nur Standardbibliothek. Kein Installationsschritt:

```
python transcript_archiver.py --version
python transcript_archiver.py --help
```

1. Schnellstart

eine Claude-Code-Sitzung als HTML archivieren (das Standardformat)

```
python transcript_archiver.py <session-id>
```

alle Formate auf einmal

```
python transcript_archiver.py <session-id> --format html,text,markdown,latex,pdf
```

die Indexseite von allem auf der Festplatte neu erzeugen

```
python transcript_archiver.py --index
```

am mitgelieferten Schaugespräch ausprobieren

```
python transcript_archiver.py 0000c0de-cafe-4000-8000-00000000f00d \  
  --projects-root examples --archive-dir demo --format html,markdown,pdf
```

Die Sitzungs-ID ist der Name der .jsonl-Datei unter ~/.claude/projects/<project>/. --index listet jede Sitzung, die es finden kann, mit ID und Titel — führen Sie es zuerst aus, wenn Sie die ID nicht kennen.

2. Quellen

Quelle	Wie	Anmerkungen
Claude Code CLI / Desktop-App	Vorgabe; Sitzungen unter --projects-root (~/.claude/projects)	natives Format
Claude Code Web/Mobil, zu Ihrem Rechner überbrückt	dasselbe	Brückendatensätze werden in der Kette zu einem Gespräch aufgelöst
Cowork von Claude Desktop (lokaler Agentenmodus)	--cowork-root (je Plattform automatisch erkannt)	gleiches Datensatzschema, anderes Basisverzeichnis; audit.jsonl wird übersprungen. Nur gegen synthetische Daten geprüft
claude.ai-Chats, Claude-Desktop-Chat, mobile App	--import-claude-ai conversations.json	aus Settings → Privacy → Export data. Nur eigenständige Chats; keine Projektgespräche, keine Verbrauchs- oder Modelldaten (die Seite sagt das)
Claude-Code-Cloud-Sitzungen ohne Brücke	nicht archivierbar	auf Ihre Festplatte wird nichts geschrieben

Automatische Cowork-Erkennung: %APPDATA%\Claude\local-agent-mode-sessions unter Windows, ~/Library/Application Support/Claude/local-agent-mode-sessions unter macOS, ~/.config/Claude/local-agent-mode-sessions unter Linux. --cowork-root "" schaltet sie ab.

3. Referenz der Kommandozeile

Jede Ein- und Ausgabe ist über die Kommandozeile erreichbar; nichts ist fest verdrahtet. --help druckt jede Option mit ihrer Vorgabe.

Positionsargument

session_id	UUID des Transkripts (der .jsonl-Dateiname). Bei --index oder --import-claude-ai optional.
------------	---

Auffinden und Ablage

Option	Vorgabe	Bedeutung
<code>--projects-root DIR</code> <code>--cowork-root DIR</code>	<code>~/.claude/projects</code> je Plattform automatisch	wo Claude Code Sitzungen schreibt <i>Cowork</i> -Sitzungen von Claude Desktop, in die Erkennung einbezogen, wenn das Verzeichnis existiert; "" schaltet ab
<code>--archive-dir DIR</code> <code>--out PATH</code>	<code>\$CLAUDE_ARCHIVE_DIR</code> oder <code>~/claude-archives</code> —	wohin Archive, <code>index.html</code> und <code>logs/</code> gehen Ausgabepfad- Stamm für ein einzelnes Archiv; jedes Format hängt seine eigene Endung an (<code>--out report.pdf --format html</code> schreibt <code>report.html</code>). Überschreibt die Benennung durch <code>--archive-dir</code>
<code>--title TEXT</code>	der eigene <code>ai-title</code> der Sitzung	Seitentitel; bestimmt auch den Dateinamens- <i>Slug</i> . Erneutes Archivieren mit anderem Titel schreibt eine neue Datei
<code>--summary-file FILE</code>	Platzhalter	HTML-Fragment (<code>h3</code> / <code>ul</code> -Blöcke), das als handgeschriebene Sitzungszusammenfassung dargestellt wird

Inhalt

Option	Vorgabe	Bedeutung
<code>--format LIST</code>	<code>html</code>	kommagetrennt: <code>html</code> , <code>text</code> , <code>markdown</code> (oder <code>md</code>), <code>latex</code> , <code>pdf</code>
<code>--tool-output on\ off</code>	<code>on</code>	Werkzeug-Ein- und -Ausgabe einbeziehen. Unabhängig von <code>--format</code> . <code>off</code> reduziert jeden Werkzeugaufruf auf eine beschriftete Zeile — meist das, was man für LaTeX/PDF will
<code>--max-tool-output N</code>	<code>16384</code>	die Mitte jeder Werkzeugausgabe über N Zeichen auslassen; jede Auslassung wird auf der Seite gezählt. 0 = nie
<code>--full</code>	<code>aus</code>	nie auslassen (wie <code>--max-tool-output 0</code>)
<code>--subagents on\ off</code>	<code>on</code>	Subagenten-Transkripte als Anhangsabschnitte darstellen. Mit <code>off</code> bleiben sie im Treuebericht aufgeführt und ihr Verbrauch zählt weiter
<code>--no-follow-chain</code>	<code>aus</code>	genau die angegebene ID archivieren, auch wenn eine vollständigere Fortsetzung existiert

Option	Vorgabe	Bedeutung
<code>--fragment</code>	aus	mit <code>--format latex</code> : nur der Rumpf, keine Präambel, transliteriert, damit er unter pdf _l atex wie unter XeLaTeX kompiliert. Nicht mit pdf kombinierbar
<code>--paginate N</code>	0	teilt das HTML in Seiten zu N Beiträgen; Seite 1 behält Zusammenfassung, Verbrauch und Treuebericht; die Seitenleiste verlinkt über die Seiten
<code>--lang CODE</code>	<code>\$CLAUDE_ARCHIVE_LANG</code> oder <code>en</code>	<code>en</code> , <code>pt-BR</code> , <code>es</code> , <code>de</code> , <code>fr</code> : die Sprache der eigenen Worte des Archivars in jedem Format und im Index. Das Gespräch wird nie übersetzt (siehe §4, <i>Sprache</i>)

Index

Option	Bedeutung
<code>--index</code>	<code>index.html</code> in <code>--archive-dir</code> neu erzeugen und beenden
<code>--watch SECONDS</code>	mit <code>--index</code> : alle SECONDS (mindestens 30) neu erzeugen, bis Ctrl+C, und die Seite zum Selbstnachladen markieren; beim Anhalten wird der Index noch einmal geschrieben, sodass er sich nicht mehr nachlädt

claude.ai-Import

Option	Bedeutung
<code>--import-claude-ai FILE</code>	Gespräche aus einer <code>claude.ai-conversations.json</code> importieren
<code>--conversation TEXT</code>	nur Gespräche, deren Name oder UUID TEXT enthält (ohne Beachtung der Groß-/Kleinschreibung)
<code>--list-conversations</code>	die Gespräche des Exports auflisten und beenden

Ausgabesteuerung und Protokollierung

Option	Bedeutung
<code>--verbose</code>	Detail je Schritt (analysierte Dateien, Kompilierdurchläufe, Pfad des Prüfprotokolls)
<code>--quiet</code>	nichts außer Warnungen ausgeben; das Prüfprotokoll erfasst weiterhin alles
<code>--log-dir DIR</code>	wohin das Prüfprotokoll je Lauf geht (Vorgabe <code><archive-dir>/logs/</code>)
<code>--version</code>	Version des Archivars ausgeben und beenden
<code>--help</code>	Optionsreferenz

Ungültige Kombinationen werden abgelehnt, bevor etwas geschrieben wird: `--watch` ohne `--index`; `--conversation/--list-conversations` ohne `--import-claude-ai`; `--fragment` ohne `latex` oder zusammen mit `pdf`; `--verbose` mit `--quiet`; ein unbekannter `--format`-Wert.

4. Was erzeugt wird

Dateien

In `--archive-dir` (oder am `--out-Stamm`), eine Datei je Format: `<session-id>_<title-slug>.html|.txt|.md|.tex|.pdf`. Ein LaTeX-Rumpf aus `--fragment` heißt `<stem>_fragment.tex`. Paginiertes HTML ergänzt `<stem>_p2.html`, `<stem>_p3.html`, ... `claude.ai`-Importe heißen `<uuid-prefix>_<slug>`. `--index` schreibt `index.html`. Jeder Lauf schreibt `logs/<timestamp>_<label>.log`.

Ist eine Sitzung ein fortgesetztes oder überbrücktes Gespräch, wird die Datei nach dem tatsächlich archivierten Transkript benannt (der vollständigsten Datei der Kette), und die Seite hält fest, welche ID angefragt wurde.

Die Seite

Jedes Format trägt, in dieser Reihenfolge: die **Sitzungszusammenfassung** (handgeschrieben über `--summary-file`, sonst ein Platzhalter), **Verbrauch und Kosten**, den **Treuebericht**, dann das **Transkript**, dann **Subagenten-Transkripte** als Anhänge.

Beitragsarten und wie jedes Format sie zeigt:

Beitrag	HTML	Text / Markdown	LaTeX / PDF
Menschlicher Prompt (P_n)	rechtsbündige Sprechblase, wortgetreu, diktengleich bei Spalten, URLs verlinkt	wortgetreu, nie neu umbrochen (Markdown: eingezäunt)	wortgetreuer Kasten
Claudes Antwort (R_n)	Markdown gerendert	Prosa neu umbrochen (Markdown: lebendes Markdown)	Markdown $\rightarrow$ LaTeX
Denken	eingeklappt; in der Praxis leer (siehe §7)	beschriftet	beschrifteter Kasten
Werkzeugaufruf	eingeklappte Ein-/Ausgabe, Fehler- und Wartezustände, Bildschirmfotos	vollständige Ein-/Ausgabe oder eine Zeile (<code>--tool-output</code>)	vollständige Ein-/Ausgabe oder nur Titeltasten
Eingefügtes Bild	eingebettet	als weggelassen angekündigt	als weggelassen angekündigt
Harness / System / Ereignis	eingeklappte Spur mit dem Klassifikationsbeleg	beschriftete Blöcke	beschriftete Kästen
Subagenten-Transkript	ausklappbarer Anhang, verlinkt vom erzeugenden Aufruf	Anhangsabschnitt	Anhangsabschnitt

Sprache

`--lang pt-BR|es|de|fr` (oder die Umgebungsvariable `CLAUDE_ARCHIVE_LANG`; die Option gewinnt; Vorgabe `en`) legt die Sprache von allem fest, was der Archivar selbst schreibt: das Seitengerüst und seine Bedienelemente, die Beitragsbeschriftungen, die Sitzungsangaben, die Hinweise zu Verbrauch und Kosten, den Treuebericht, den Subagenten-Anhang, die Formathinweise der Text-, Markdown- und LaTeX-Ausgaben und die Indexseite. `<html lang>` und das eingebettete Metadatenfeld `lang` halten die Wahl fest; das eigenständige LaTeX lädt `polyglossia`, wenn es installiert ist, behält **Englisch als Standardsprache** — die Prosa des Gesprächs wird wie Englisch getrennt und gesetzt, sodass eine französische Seite nie Leerzeichen vor Claudes ! einfügt — und hüllt nur die eigenen Worte des Archivars in die Dokumentsprache. Ein `--fragment` setzt diese Worte mit Akzentmakros

(\{'e'}, \{"a"}, \ss{})), damit pdf_latex sie unversehrt druckt, und zählt sie nie in der Verlusthinweis des Fragments mit.

Das Gespräch wird nie übersetzt. Prompts, Antworten, Denken, Werkzeugnamen, Werkzeug-Ein- und -Ausgabe, System- und Harness-Text, Modellnamen, Titel, Pfade, Daten (ISO) und Zahlen sind in jeder Sprache dieselben Bytes — die Suite rendert das Prüfmuster in allen fünf Sprachen und prüft, dass jedes Gesprächsfragment der englischen Seite in den anderen wortgetreu vorhanden ist. Ereignisabzeichen und Anhangsbeschriftungen werden dort übersetzt, wo sie dargestellt werden; die Datensatztypnamen in den Treuetabellen (`human turn`, `tool use`, ...) sind Parser-Vokabular und bleiben englisch, ebenso der Stempel `archiver v...`, den der Index zurückliest. Prüfprotokoll, Konsole (`--verbose`) und `--help` bleiben in jeder Sprache englisch. Ein unbekannter Code — in der Option oder in der Variablen — wird abgelehnt, bevor etwas geschrieben wird.

Referenzmarken

Jeder menschliche Prompt ist P1, P2, ... und jede Antwort R1, R2, ..., fortlaufend innerhalb des Dokuments; Beiträge von Subagenten erhalten das Präfix A1., A2. (also A2.R4). Im HTML sind die Marken Anker: `page.html#P32` verlinkt direkt auf den Prompt.

Treuebericht

Jeder Quelldatensatz wird **dargestellt** (erzeugte einen oder mehrere Beiträge), **eingefaltet** (ein Werkzeugergebnis, das in seinen Aufruf aufgenommen wurde) oder **gezählt** (Metadaten ohne Transkriptinhalt — und beschädigte Zeilen). Die drei Zahlen werden auf der Seite gegen die Datensatzzahl der Quelle abgeglichen; gehen sie nicht auf, sagt die Seite das, statt es zu verbergen. Der Bericht listet außerdem die Datensätze nach Typ, die Inhaltsblöcke, was dargestellt und was gezählt wurde, den Beleg menschlich-gegen-eingespielt je Datensatz, die Subagentendateien und die Vorbehalte (leere Denkböcke, unaufgelöste Werkzeugaufrufe, Zeitpunkt der Momentaufnahme gegenüber dem letzten Datensatz der Quelle).

Verbrauch und Kosten

Tokens je Modell, dedupliziert nach `requestId` — eine API-Antwort wird als mehrere Datensätze geschrieben, die denselben Verbrauch wiederholen, und sie zu summieren überschätzt die Ausgabe bei werkzeuglastigen Sitzungen um ~2,3×. Cache-Lesevorgänge sowie 5-Minuten- und 1-Stunden-Cache-Schreibvorgänge werden getrennt, und die Kosten werden zu **öffentlichen Listenpreisen** aus der PRICING-Tabelle am Kopf des Skripts geschätzt (Cache-Lesevorgänge zu 0,1× der Eingabe, Schreibvorgänge zu 1,25× / 2×). Das ist nicht, was ein Abonnement in Rechnung stellt. Modelle, die die Tabelle nicht kennt, werden als „kein Listenpreis“ gemeldet. Der Verbrauch von Subagenten wird eingerechnet.

Gemeldete Kosten. Claude Code 2.1.9x schreibt auch seinen eigenen Zähler in die Sitzungsdatei (`cost-state`-Datensätze: laufende Kosten, Kosten je Modell, von Werkzeugen hinzugefügte und entfernte Zeilen). Ist er vorhanden, zeigt die Seite diese Zahl als Spalte *gemeldete Kosten* neben der Listenschätzung, als Zeile in den Sitzungsangaben und als `reported_cost_usd`, `reported_cost_runs`, `reported_cost_partial`, `lines_added`, `lines_removed` in den eingebetteten Metadaten; die Formate Text, Markdown und LaTeX tragen denselben Satz. Der Zähler gilt **je Prozess**: Jedes `claude --resume` startet einen neuen Zähler, und Läufe vor der Existenz des Datensatzes schreiben keinen — die Zahl ist also die Summe der letzten Momentaufnahme jedes Laufs (aus allen Dateien der Kette einer fortgesetzten Sitzung zusammengetragen) und wird als **teilweise** gekennzeichnet, wenn die Sitzung mehr als eine Minute vor ihrem ersten gemessenen Lauf begann. Dann nennt die Seite, welche Ausgaben nicht abgedeckt sind, und der Index zeigt weiter die Listenschätzung; sonst zeigt der Index „\$X gemeldet“. Ein Lauf, den Claude Code nicht vollständig bepreisen konnte, wird vermerkt („die gemeldete Summe ist eine Untergrenze“). In der Praxis lag der Zähler bei einer Sitzung mit einem Lauf ~30 % unter der Listenschätzung.

Die Bedienelemente der HTML-Seite

Seitenleiste: **Suche** (blendet Beiträge aus, deren Text nicht passt), **Filter** (grenzt die Inhaltsliste ein; Taste /), Spur-Schalter (Denken, Werkzeuge, Harness, Ereignisse, Subagenten), alles auf-/zuklappen, **Themenschalter** (hell oder dunkel, je Browser gemerkt; folgt dem Betriebssystem, bis Sie wählen), Sitzungsangaben, Inhalt. Tasten: j/k springen zwischen menschlichen Beiträgen.

Eine **Schutzverweigerung mit Modellwechsel** (Claude Code schreibt einen `system/model_refusal_fallback`-Datensatz, wenn eine Nachricht abgelehnt wird und die Sitzung mit einem anderen Modell weitergeht) wird in jedem Format als Ereignis dargestellt: das Abzeichen *Model fallback after a safeguard refusal*, das Detail `<original> -> <fallback> (category: ...)`, N `message(s) retracted` und ein Rumpf, der angibt, wie viele der zurückgezogenen Nachrichten in der Quelldatei fehlen. Die HTML-Sitzungsangaben ergänzen eine Zeile *Harness retractions*. Die Rückschau, die Claude Code bei Ihrer Rückkehr druckt (`away_summary`), ist das Ereignis *Away summary*.

Der Index

`--index` durchsucht jede Sitzung auf der Festplatte und markiert sie als **archiviert**, **veraltet** (die Quelle hat neuere Datensätze als das Archiv), **abgedeckt** (in ein anderes, archiviertes Transkript fortgesetzt), **Alt-v1** oder **nicht archiviert**; listet Archive, deren Quelle nicht auf der Festplatte liegt (claude.ai-Importe, gelöschte Transkripte); und zeigt eine Aktivitätsspalte, deren Altersangaben im Browser altern. Die Kopfzeilen sortieren per Klick. `--watch` erzeugt ihn fortlaufend neu: Jede geschriebene Seite trägt ein `<meta http-equiv="refresh">`, damit ein offener Browser mitgeht. Wenn die Beobachtung endet — Ctrl+C, ein geschlossenes Konsolenfenster, ein `taskkill` auf die PID — wird der Index noch einmal ohne diese Marke geschrieben, sodass eine offen gelassene Seite nicht länger alle N Sekunden einen eingefrorenen Index nachlädt. Kann dieses letzte Schreiben nicht erfolgen (Datei gesperrt, Festplatte voll), sagt der Lauf das und nennt, was noch auf der Festplatte liegt; führen Sie `--index` erneut aus, um ihn zu ersetzen. Ein erstes `--index` in ein noch nicht existierendes Verzeichnis legt dieses an.

Suche über alle Archive. Die Indexseite trägt ein Suchfeld über jeden menschlichen Prompt jedes Archivs — alle Seiten eines paginierten Archivs und Subagenten-Prompts (A1.P1) eingeschlossen —, beim Indizieren aus dem eigenen HTML der Archive zurückgelesen, sodass Archive früherer Versionen und claude.ai-Importe gleichermaßen erfasst sind. Ab zwei eingegebenen Zeichen werden die passenden Prompts aufgelistet (Sitzung, Marke, Titel, hervorgehobener Ausschnitt; die ersten 200 werden gezeigt), jeder mit direktem Link auf den Anker des Prompts auf seiner Seite, und die Sitzungstabelle wird auf die Treffer eingegrenzt. Prompts sind im Index auf 400 Zeichen begrenzt; Claudes Antworten sind innerhalb jeder Seite durchsuchbar, nicht über Archive hinweg (siehe Einschränkungen).

5. LaTeX und PDF

Voraussetzungen: eine TeX-Installation, die `xelatex`, `fvextra`, `tcolorbox`, `booktabs`, `array`, `enumitem`, `xcolor`, `hyperref` und die DejaVu-Schriften bereitstellt (TeX Lives `scheme-full` hat alles davon). Schriften werden **nach Dateiname aus TeX Live** geladen, nicht aus dem System, sodass die Ausgabe nicht von der Schriftdatenbank des Rechners abhängt.

- `pdf` = das eigenständige LaTeX, zweimal von `xelatex` kompiliert (wegen des Inhaltsverzeichnisses); `.aux/.log/.out/.toc` werden bei Erfolg entfernt und die `.tex` nur behalten, wenn `latex` ebenfalls angefordert war. Bei einem Fehlschlag werden die letzten 30 Zeilen des Protokolls gedruckt und die `.tex` bleibt zur Untersuchung liegen.
- `--fragment` gibt einen Rumpf zum `\input` in Ihr eigenes Dokument aus. Er ist engine-neutral: Griechisch wird zu Mathematik ($\Gamma \rightarrow \$\Gamma$), Tief- und Hochstellungen werden zu Mathematik, Pfeile und Rahmenzeichen zu ASCII, Akzente werden auf den Grundbuchstaben reduziert. Ihre Präambel braucht `\usepackage{fvextra} \usepackage{xcolor} \usepackage{enumitem} \usepackage{booktabs} \usepackage{array} \usepackage[most]{tcolorbox}`. Die Beitragsumgebungen sind mit `\ifundefined` definiert, sodass Sie sie aus Ihrer Präambel umgestalten können.
- Emoji und andere Glyphen, die keine TeX-Schrift setzen kann, sowie C0/C1-Steuerbytes (NULs aus UTF-16-Konsolenmitschnitten, Rückschritte), werden entfernt und **im Dokument gezählt**. Zeilen über 500 Zeichen werden hart umbrochen, damit TeX sie setzen kann; die Anzahl wird genannt.
- Ein Beitrag länger als 1.500 gesetzte Zeilen (ein riesiger Einfügeblock oder eine Werkzeugausgabe) wird in aufeinanderfolgende Kästen mit dem Titel (*part k/n*) geteilt: Ein einziger umbruchfähiger Kasten, der ihn ganz enthält, erschöpft TeXs Speicher. Das Dokument nennt, wie viele Beiträge geteilt wurden; nichts wird weggelassen.
- **Markdown-Tabellen werden in Stücke von höchstens 30 gesetzten Zeilen zerlegt**, jedes ein eigenes `tabular`, das die Kopfzeile wiederholt und mit (*table continued*) gekennzeichnet ist, denn ein

einzelnes `tabular` kann nicht über eine Seite hinweg umbrechen. Eine Tabelle, deren natürliche Breite die Zeile übersteigt, erhält gleichmäßig umbrechende `p`-Spalten statt natürlicher, sodass keine Zelle über das Papier hinausläuft. Beides waren vor 2.6.4 stille Verluste.

- Bestätigt durch einen vollständigen Durchlauf über ein echtes Archiv mit 64 Sitzungen (6.245 Seiten, 69 Minuten, `--tool-output off`, 64/64 kompiliert, August 2026) und durch eine Kompilier-und-Zähl-Prüfung in der Suite: Eine Antwort, die eine Tabelle mit 300 Zeilen ist, muss die Seiten belegen, die ihre Zeilen brauchen, und nicht bloß mit 0 enden.
- Kosten der vollständigen Werkzeug-Ein-/Ausgabe, gemessen: eine Sitzung mit 636 Datensätzen → 643 Seiten in etwa vier Minuten; eine Sitzung mit 1.655 Datensätzen umfasst 92 Seiten mit `--tool-output off` und 260 mit ihr.

6. Protokollierung und Prüfung

Konsole: standardmäßig Fortschrittszeilen; `--quiet` schaltet sie stumm; `--verbose` ergänzt Detail je Schritt. Warnungen gehen immer nach `stderr`. Jeder Aufruf schreibt `<archive-dir>/logs/<YYYYMMDD-HHMMSS>_<label>.log` (oder unter `--log-dir`) mit den Versionen von Archivar und Python, der exakten Kommandozeile, dem Arbeitsverzeichnis, Start- und Endzeit, jeder Konsolenmeldung und dem Ausgang (`ok`, `failed: ...`, `crashed: ...`, `interrupted`). Die Protokollierung bricht einen Lauf nie ab.

7. Bekannte Einschränkungen

Das sind die ehrlichen Kanten. Jede wird auf der Seite genannt, auf der sie gilt.

- **Der Denktex steht nie im Transkript.** Claude Code fordert das Denken mit `display: "omitted"` an; jeder Denkblock auf der Festplatte ist leer. Das Archiv zeigt, *dass* Claude an einer Stelle gedacht hat, nie was.
- **Die Listenkosten sind eine Schätzung**, keine Rechnung; die `PRICING`-Tabelle ist fest eingetragen (Preise von August 2026) und muss bei Preisänderungen bearbeitet werden. Die **gemeldeten Kosten** sind Claude Codes eigene Zahl, gelten aber je Prozess: Über mehrere Läufe fortgesetzte Sitzungen oder solche, die vor Claude Code 2.1.9x begannen, sind nur teilweise erfasst und sagen das (`partial`).
- **Eine laufende Sitzung ist um einen Werkzeugaufruf versetzt:** Archivieren aus der Sitzung heraus lässt den eigenen Aufruf des Archivars unaufgelöst; die Seite sagt das.
- **Der `claude.ai`-Export enthält nur eigenständige Chats** — keine Projektgespräche, keine *Cowork*-Sitzungen, keinen Verbrauch und keine Modellnamen. Er zielt auf das Exportschema von Mitte 2026.
- **Die *Cowork*-Erkennung folgt dem dokumentierten Aufbau**, wurde aber nur gegen synthetische Daten geprüft; der lokale *Cowork*-Speicher übersteht eine Neuinstallation der App nicht.
- **Cloud-Sitzungen ohne Brücke zu Ihrem Rechner lassen sich nicht archivieren.**
- **Text und Markdown können keine Bilder tragen;** sie werden als weggelassen angekündigt. LaTeX/PDF ebenso; das HTML enthält sie.
- **Die Markdown-Darstellung deckt Claudes eigene Prosa ab** (Überschriften, Listen inkl. verschachtelter, Tabellen, Codeblöcke beliebiger Länge, Zitate, Inline-Code/fett/kursiv/durchgestrichen/Links), nicht beliebiges CommonMark: kein HTML-Durchreichen, keine Referenzlinks, keine Fußnoten; Tabellenzellen werden an jedem `|` getrennt. In LaTeX und PDF wird eine Tabelle zerlegt und, wenn breit, umbrochen (§5): Alle Zellen überleben, aber eine sehr breite Tabelle wird spaltenweise ausgeglichen statt nach Geschmack gesetzt.
- **Die Einstufung menschlich-gegen-eingespielt** ist bei Datensätzen mit `promptSource / origin.kind` maßgeblich; ältere Datensätze greifen auf Textmarker zurück, und der verwendete Beleg wird je Datensatz im Treuebericht aufgeführt.
- **Zeitstempel sind lokal** zur archivierenden Maschine (im HTML für UTC mit der Maus darüberfahren).
- **Erneutes Archivieren mit anderem `--title` schreibt eine neue Datei** neben die alte, statt sie zu überschreiben.
- **Skalierung:** Jeder Lauf liest alle `.jsonl`-Dateien unter den Wurzeln erneut, um Ketten aufzulösen; der Index vergleicht UUID-Mengen paarweise. Gut für Hunderte von Sitzungen; langsam für Tausende.
- **Plattformen:** entwickelt und geprüft unter Windows; die Suite und eine statische `pyflakes`-Prüfung laufen in der CI unter Linux, Windows und macOS. Linux hatte einen echten Lauf (31.08.2026, WSL2 Ubuntu, Python 3.14: alle Nicht-PDF-Formate einer echten Sitzung, `--index` und das laute Scheitern ohne `xelatex`).

Im Feld unbestätigt: PDF und TeX-Schriftpfade unter Linux, unter Linux erzeugte *Cowork*-Sitzungen und macOS insgesamt. Unter WSL machte das Lesen der Transkripte über `/mnt/c` den Scan etwa viermal langsamer als nativ (18 s gegenüber 4 s für 281 Transkripte) — halten Sie die Wurzeln auf der Linux-Seite.

- **Der Aktivitätsverfall im Live-Index ist einseitig:** Eine Sitzung kann auf dem Bildschirm still werden, aber ohne Neuerzeugung (`--watch`) nicht wieder aktiv.
- **Die archivübergreifende Suche erfasst Prompts, nicht Antworten** (und die ersten 400 Zeichen jedes Prompts). Antworten sind innerhalb einer Seite durchsuchbar. Auch die Antworten zu indizieren würde die Indexdatei vervielfachen und ist zurückgestellt.

8. Tests

```
python tests/test_archiver.py
```

528 Prüfungen gegen die synthetischen Sitzungen in `examples/` (kein echtes Transkript nötig). LaTeX-/PDF-Kompilierprüfungen werden übersprungen, nicht als Fehler gewertet, wenn kein TeX im PATH liegt. Um es an einem eigenen Gespräch zu erproben:

```
CLAUDE_PROJECTS=~/.claude/projects SAMPLE_SESSION=<id> python tests/test_archiver.py
```

Die Suite prüft außerdem, dass dieses Handbuch und `AGENTS.md` jede Kommandozeilenoption dokumentieren und dass die im README genannte Prüfungszahl aktuell ist.

9. Dieses Handbuch bauen

```
python docs/build_manual.py
```

Rendert `USER_MANUAL.md` — und jede Übersetzung — nach `.html` und `.pdf` mit pandoc (und xelatex für das PDF), wenn verfügbar, sonst mit dem eigenen Markdown-Renderer des Archivars für das HTML und einem Hinweis, dass das PDF übersprungen wurde. Die gebauten Dateien sind eingeecheckt, damit Leser kein Werkzeug brauchen.

Englisch ist der Referenztext. Jede Übersetzung hält den Digest des englischen Textes fest, aus dem sie entstand, und die Suite schlägt fehl, wenn das Englische sich bewegt hat und die Übersetzung nicht; `python docs/build_manual.py --stamp` schreibt diese Digests — nachdem eine Übersetzung nachgeführt wurde, nie statt dessen.